

TRABAJO PRÁCTICO INTEGRADOR

PROGRAMACIÓN II

Food Store – Sistema de Gestión de Pedidos de
Comida (Consola +JDBC)

Repositorio del proyecto

<https://github.com/magaagea1/IntegradorProg2>

Video

[Trabajo Práctico Integrador – Programación II.mp4 - Google Drive](#)

UML y Capturas de Pantalla

[IntegradorProg2/UML y Capturas de pantalla at master · magaagea1/IntegradorProg2](#)

Dominicale Doré, María Luz

Egea Ruiz, Magaly

Ferrario, Inés

Fecha límite de entrega: 22/06/2026

2. Índice

3. Marco teórico.....	2
4. Decisiones Técnicas y Arquitectura	3
5. Dificultades y Soluciones	7
6. Bibliografía / Webgrafía	8

3. Marco teórico

3.1. Java

Java es un lenguaje de programación orientado a objetos ampliamente utilizado para el desarrollo de aplicaciones de distintos tipos. En este proyecto se utilizó **Java 21** junto con el entorno de desarrollo **NetBeans**, implementando una aplicación de consola basada en menús y entrada de datos por teclado.

3.2. Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos (POO) permite modelar entidades del mundo real mediante clases y objetos. En el desarrollo se aplicaron conceptos fundamentales como:

- **Encapsulamiento:** mediante atributos privados y métodos de acceso.
- **Herencia:** utilizando una clase base común para las entidades.
- **Abstracción:** definiendo características compartidas entre objetos.
- **Polimorfismo:** mediante la sobrescritura de métodos y el uso de interfaces.

3.3. Colecciones e interfaces

Para el almacenamiento de información se utilizaron colecciones **ArrayList** (datos en memoria durante ejecución). Se implementó la interfaz **Calculable**, utilizada para definir el cálculo automático del total de los pedidos y una clase de utilidades destinada a centralizar validaciones comunes, aplicando el principio DRY (Don't Repeat Yourself).

3.4. Enumeraciones y Excepciones

El sistema utiliza enumeraciones (Enums) para representar valores predefinidos como roles, estados de pedido y formas de pago, asegurando consistencia en los datos.

También se implementaron excepciones personalizadas para validar reglas de negocio y manejar errores de forma clara y segura durante la ejecución.

3.5. UML

El diagrama UML provisto por la cátedra sirvió como base para modelar las entidades, relaciones 1..N, composición y la interfaz Calculable. Este modelo guió la estructura del código y la organización de las clases.

4. Decisiones Técnicas y Arquitectura

4.1. Enfoque general de la solución

El sistema Food Store fue desarrollado como una aplicación de consola en Java 21, siguiendo principios de POO y el diagrama UML provisto por la cátedra. La solución funciona íntegramente en memoria mediante colecciones dinámicas y mantiene una clara separación de responsabilidades entre capas. Su objetivo es ofrecer una aplicación modular, mantenible y alineada con buenas prácticas de POO.

4.2. Arquitectura por capas

Capa de dominio (entities/)

Contiene las clases del modelo según el UML: Base (abstracta), Categoria, Producto, Usuario, Pedido, DetallePedido.

La clase Pedido implementa la interfaz Calculable, obligando a definir el método `calcularTotal()`.

Capa de lógica de negocio (service/)

Implementa los servicios responsables de: CRUD de cada entidad, validaciones, reglas de negocio, búsquedas en colecciones, unicidad de email, baja lógica, integridad del stock, composición Pedido–DetallePedido. Esta capa evita que el menú contenga lógica compleja.

Capa de interacción con el usuario (main/)

Incluye: menú principal, submenús CRUD, lectura de datos mediante Scanner comunicación con los servicios, Regla aplicada: “El Main no debe contener reglas de negocio.”,

Capa de utilidades (utils/)

Incluye validaciones reutilizables: cadenas no vacías, números positivos, formato básico de email, manejo de excepciones de entrada

Capa de excepciones (exception/)

Excepciones personalizadas: `DatoInvalidoException`, `EntidadNoEncontradaException`, `OperacionNoPermitidaException`, `StockInsuficienteException`.

Permiten manejar errores sin interrumpir la ejecución del programa.

Capa de enumeraciones (enums/)

Incluye:

- Rol (ADMIN, USUARIO)
- Estado (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO)
- FormaPago (TARJETA, TRANSFERENCIA, EFECTIVO)

4.3. Principales decisiones técnicas

- Utilizar una clase abstracta *Base* para centralizar atributos comunes.
- Implementar baja lógica mediante el atributo **eliminado**.
- Utilizar *ArrayList* para almacenar entidades en memoria.
- Implementar composición entre *Pedido* y *DetallePedido*.
- Aplicar validaciones estrictas antes de crear o modificar entidades.
- Centralizar las validaciones en una clase de utilidades para aplicar el principio DRY y evitar duplicación de código.
- Implementar excepciones personalizadas para evitar cierres inesperados.
- Utilizar la interfaz *Calculable* para garantizar el cálculo del total del pedido.
- Separar la lógica del menú de la lógica de negocio.
- Mantener un `toString()` útil en cada entidad para facilitar los listados en consola.

4.4. Modelo UML ([Link](#) a UML tamaño más grande y otras imágenes)

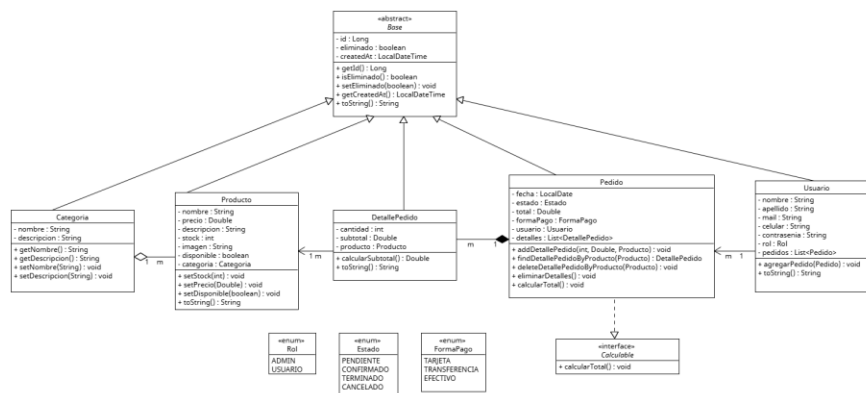


Diagrama UML del sistema Food Store (Para ver en mayor tamaño ver en el anexo). El diagrama presenta el modelo de clases utilizado en la aplicación. La clase abstracta Base centraliza atributos comunes y es extendida por Categoría, Producto, Usuario, Pedido y DetallePedido. La clase Pedido implementa la interfaz Calculable para el cálculo del total, y mantiene una relación de composición con DetallePedido. Las asociaciones entre entidades reflejan las relaciones 1..N definidas en el dominio, mientras que los enumerados Rol, Estado y FormaPago representan valores constantes del sistema.

4.5. Funcionamiento del Sistema

A continuación, se detalla el funcionamiento del sistema según los módulos del menú principal. Food Store es una aplicación de consola en Java que permite gestionar categorías, productos, usuarios y pedidos mediante un menú que organiza las operaciones CRUD y aplica reglas de negocio para mantener la integridad de los datos.

4.5.1. Menú principal

Al iniciar la aplicación, el usuario visualiza un menú con opciones para gestionar categorías, productos usuarios, pedidos y salir del sistema

Cada opción redirige a un submenú específico. El menú se ejecuta en un bucle hasta que el usuario selecciona Salir.

4.5.2. CRUD de Categorías

Permite: ● Crear nuevas categorías ● Listar categorías existentes ● Modificar nombre o descripción ● Eliminar categorías (baja lógica)	Reglas de negocio: ● No se permiten nombres vacíos. ● No se puede eliminar una categoría con productos asociados. ● La eliminación es lógica mediante eliminado.
---	---

4.5.3. CRUD de Productos

Permite: ● Crear productos asociados a una categoría ● Listar productos	Reglas de negocio: ● El precio debe ser mayor a cero. ● El stock no puede ser negativo. ● Un producto debe pertenecer a
--	---

● Modificar precio, stock o descripción ● Eliminar productos (baja lógica)	una categoría válida. ● No se puede eliminar un producto asociado a un pedido activo.
---	--

4.5.4. CRUD de Usuarios

Permite: ● Crear usuarios ● Listar usuarios ● Modificar datos personales ● Eliminar usuarios (baja lógica)	Reglas de negocio: ● El email debe tener formato válido. ● No se permiten emails duplicados. ● No se puede eliminar un usuario con pedidos activos.
---	---

4.5.5. Gestión de Pedidos

Permite: ● Crear un pedido ● Agregar productos ● Modificar cantidades ● Eliminar productos ● Confirmar o cancelar ● Listar pedidos por usuario ● Calcular el total automáticamente	Reglas de negocio: ● No se puede agregar un producto sin stock. ● Si se agrega un producto repetido, se actualiza la cantidad. ● El total se calcula mediante la interfaz Calculable. ● Un pedido confirmado no puede modificarse. ● La composición garantiza que al eliminar un pedido se eliminan sus detalles.
--	---

4.5.6. Validaciones

Centralizadas en *utils*: cadenas no vacías, números positivos, formato de email, manejo de excepciones de entrada

4.5.7. Excepciones personalizadas

Cada una muestra mensajes claros para guiar al usuario, son cuatro en total:

`DatoInvalidoException`, `EntidadNoEncontradaException`,
`OperacionNoPermitidaException`, `StockInsuficienteException`.

4.5.8. Servicios y lógica de negocio

Cada entidad cuenta con un servicio que encapsula: búsquedas, validaciones, reglas de negocio, CRUD, gestión de listas en memoria. Esto mantiene el *main* limpio y separa responsabilidades.

4.6. Capturas del sistema funcionando ([Link](#) a todas las capturas)

Listado de pedidos y pedido completo

```
=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado / forma de pago
4. Eliminar
5. Ver pedido completo
0. Volver
Seleccione: 1
Pedido(id=12, fecha=2026-06-18, estado=PENDIENTE, formaPago=TRANSFERENCIA, total=11000,00, detalles=3, usuario=Pedro Ruiz)
```

```
=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado / forma de pago
4. Eliminar
5. Ver pedido completo
0. Volver
Seleccione: 5
Ingrese el ID del pedido: 12

=== DETALLE COMPLETO DEL PEDIDO ===
ID: 12
Usuario: Pedro Ruiz
Estado: PENDIENTE
Forma de pago: TRANSFERENCIA
Fecha: 2026-06-18
Total: $11000.0

--- DETALLES ---
Producto: Shampoo
Cantidad: 1
Precio unitario: $2500.0
Subtotal: $2500.0
-----
Producto: Agua
Cantidad: 3
Precio unitario: $1500.0
Subtotal: $4500.0
-----
Producto: Jabón
Cantidad: 2
Precio unitario: $2000.0
Subtotal: $4000.0
-----
```

5. Dificultades y Soluciones

Durante el desarrollo del sistema surgieron diversos desafíos técnicos que requirieron ajustes en la implementación. A continuación se detallan las principales dificultades y las soluciones aplicadas.

5.1. Manejo de Scanner y entradas del usuario

Se presentaron problemas al mezclar `nextInt()` y `nextLine()`, lo que generaba saltos de línea inesperados y datos vacíos.

Solución: se unificó el uso de `nextLine()` para todas las entradas y se realizó el parseo manual de números, evitando errores de lectura.

5.2. Composición Pedido – DetallePedido

Fue necesario asegurar que los detalles se agregaran correctamente al pedido y que el total se recalculara utilizando la interfaz `Calculable`.

Solución: se implementó el método `addDetallePedido()` respetando la composición y se centralizó el cálculo del total en `calcularTotal()`.

5.3. Validaciones y excepciones personalizadas

Las primeras versiones del sistema se cerraban ante entradas inválidas o IDs inexistentes.

Solución: se crearon excepciones propias y se encapsuló su manejo en los servicios para evitar interrupciones del programa.

5.4. Integridad del stock

Al agregar detalles de pedido, era posible ingresar cantidades superiores al stock disponible.

Solución: se validó el stock antes de agregar el detalle y se descontó únicamente cuando la operación fue exitosa.

5.5. Unicidad del email en usuarios

Inicialmente no se detectaban correctamente los correos duplicados.

Solución: se implementó una búsqueda previa en la colección de usuarios y se lanzó `DatoInvalidoException` cuando correspondía.

5.6. Relación 1..N entre entidades

La relación Usuario–Pedido y Pedido–DetallePedido requería garantizar consistencia en las colecciones.

Solución: se reforzó la lógica de los servicios para validar IDs, evitar referencias inválidas y asegurar que las bajas lógicas no afectaran la integridad del modelo.

5.7. Centralización de validaciones (DRY)

Al inicio, varias validaciones estaban repetidas en distintos servicios, lo que generaba duplicación de código y riesgo de inconsistencias.

Solución: se creó una clase de utilidades (`Validaciones`) para unificar todas las reglas comunes, aplicando el principio DRY y mejorando la mantenibilidad del sistema.

6. Bibliografía / Webgrafía

- Bell, D. (2023). The UML 2 class diagram. IBM Developer. <https://developer.ibm.com>
- Universidad Tecnológica Nacional. (s.f.). U5-A4 complemento práctico: Lecciones del ejercicio integrador. Tecnicatura Universitaria en Programación a Distancia.
- Oracle. (2024). Java Platform, Standard Edition 21 Documentation: [Link](#)
- UMLetino. (2024). Herramienta UML online: [Link](#)
- StackOverflow. (s.f.). Preguntas y respuestas técnicas sobre Java y POO: [Link](#)